Lab 11

PL/pgSQL Named Blocks: Functions and Procedures

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Lab Workflow **2**

2 Database Schema & Data Generation **5**

3 Introduction to Named Blocks **7**

4 User-Defined Functions **7**
4.1 Scalar and Table-Valued Functions 7

5 Stored Procedures **8**

6 Parameter Modes and Default Behavior **9**
6.1 Implicit Default Behavior 9
6.2 Mode Definitions 9

7 Integration of Parameter Modes **9**
7.1 Function Implementation with Mixed Modes 9
7.2 Procedure Implementation with Mixed Modes 10

8 Calling Functions and Procedures **11**
8.1 Calling Scalar Functions 11
8.2 Calling Table-Valued Functions 12
8.3 Calling Functions with OUT and INOUT Parameters 12
8.4 Calling Stored Procedures 12
8.5 Calling Procedures with OUT and INOUT Parameters 13

9 Lab Tasks **14**

1 Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

1. Connect to Your Database:

Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Verify the Connection:

Verify that you are connected to the correct database with the following command:

```
\conninfo
```

Expected output:

```
Database: cse4308_12345, User: student_12345
```

3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

Load the SQL file that sets up the tables and inserts initial data.

1. Load the SQL File:

Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_11_setup.sql
```

This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Check Tables:

Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L11_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

2 Database Schema & Data Generation

Instructions: Save this code as `lab_11_setup.sql` and run it using `\i lab_11_setup.sql` in your `psql` terminal. This script generates a e-commerce schema.

```sql
-- 1. CLEANUP
DROP TABLE IF EXISTS order_items CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS restock_logs CASCADE;
DROP TABLE IF EXISTS products CASCADE;
DROP TABLE IF EXISTS customers CASCADE;

-- 2. CREATE TABLES
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    region VARCHAR(50),
    loyalty_tier VARCHAR(20) -- 'Bronze', 'Silver', 'Gold', 'Platinum'
);

CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    category VARCHAR(50),
    base_price NUMERIC(10, 2),
    stock_quantity INT
);

CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    order_date DATE,
    status VARCHAR(20), -- 'PENDING', 'PROCESSED', 'PARTIAL', 'BACKORDERED'
    total_value NUMERIC(12, 2) DEFAULT 0.00
);

CREATE TABLE order_items (
    item_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id INT REFERENCES products(product_id),
    quantity INT,
    unit_price NUMERIC(10, 2)
);

CREATE TABLE restock_logs (
    log_id SERIAL PRIMARY KEY,
    product_id INT REFERENCES products(product_id),
    restock_date DATE,
    added_quantity INT
);

-- 3. DATA GENERATION
-- Insert 1000 Customers
INSERT INTO customers (name, region, loyalty_tier)
SELECT
    'Customer_' || id,
    (ARRAY['North', 'South', 'East', 'West'])[floor(random() * 4 + 1)],
    (ARRAY['Bronze', 'Silver', 'Gold', 'Platinum'])[floor(random() * 4 + 1)]
```

```
FROM generate_series(1, 1000) AS id;

-- Insert 200 Products
INSERT INTO products (name, category, base_price, stock_quantity)
SELECT
    'Product_' || id,
    (ARRAY['Electronics', 'Clothing', 'Groceries', 'Tools', 'Books'])[floor(random() *
     5 + 1)],
    ROUND((random() * 500 + 10)::numeric, 2),
    floor(random() * 100 + 10)::INT
FROM generate_series(1, 200) AS id;

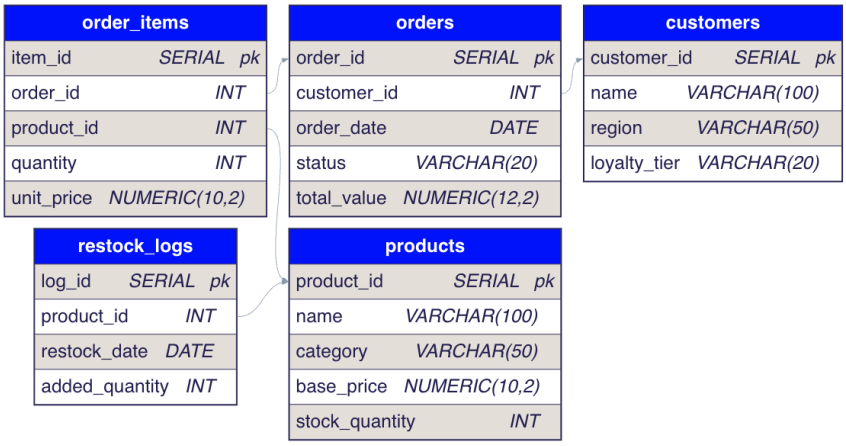
-- Insert 3000 Orders (All marked as PENDING initially)
INSERT INTO orders (customer_id, order_date, status)
SELECT
    floor(random() * 1000 + 1)::INT,
    '2026-01-01'::DATE - (floor(random() * 365)::INT),
    'PENDING'
FROM generate_series(1, 3000);

-- Insert 9000 Order Items (Averaging 3 items per order)
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT
    floor(random() * 3000 + 1)::INT,
    floor(random() * 200 + 1)::INT,
    floor(random() * 5 + 1)::INT,
    0.00 -- To be updated based on product base price
FROM generate_series(1, 9000) ON CONFLICT DO NOTHING;

-- Fix the unit_price in order_items based on products table
UPDATE order_items oi
SET unit_price = p.base_price
FROM products p
WHERE oi.product_id = p.product_id;

-- Initial aggregation to set order total_values
UPDATE orders o
SET total_value = sub.total
FROM (
    SELECT order_id, SUM(quantity * unit_price) as total
    FROM order_items
    GROUP BY order_id
) sub
WHERE o.order_id = sub.order_id;
```

Listing 1: lab_11_setup.sql



3 Introduction to Named Blocks

Anonymous blocks (`DO $$`) evaluate procedural logic for single execution but are not persistently stored in the database schema. Persistent named blocks, referred to as subprograms, encapsulate procedural logic for reuse. In PostgreSQL, subprograms are classified into two primary constructs: **User-Defined Functions** and **Stored Procedures**.

4 User-Defined Functions

A User-Defined Function is a named PL/pgSQL block that evaluates to a specific return type. Functions are invoked within expressions (e.g., a Data Manipulation Language `SELECT` statement) and structurally require a `RETURN` or `RETURN QUERY` statement. A defining characteristic of a function is that it executes entirely within the transaction scope of the invoking query; it cannot execute Transaction Control Language (TCL) commands such as `COMMIT` or `ROLLBACK`.

4.1 Scalar and Table-Valued Functions

A scalar function evaluates to a single, atomic value per invocation, utilizing the `RETURNS <datatype>` clause. Conversely, a table-valued function evaluates to a relational dataset, utilizing the `RETURNS TABLE` clause and the `RETURN QUERY` statement to populate the output relation.

```
CREATE OR REPLACE FUNCTION calculate_customer_lifetime_value(p_cust_id INT)
RETURNS NUMERIC AS $$
DECLARE
v_total_spent NUMERIC;
BEGIN
SELECT COALESCE(SUM(total_value), 0) INTO v_total_spent
FROM orders
WHERE customer_id = p_cust_id;

RETURN v_total_spent;
END;
$$ LANGUAGE plpgsql;
```

Listing 2: Scalar Function Definition

5 Stored Procedures

A Stored Procedure is a named PL/pgSQL block invoked via the `CALL` statement. Unlike functions, procedures do not evaluate to a direct return value (omitting the `RETURNS` clause) and are capable of autonomous transaction control. Procedures can initiate and terminate transactions using TCL commands (`COMMIT` and `ROLLBACK`). This capability dictates their use for bulk data manipulation, schema migrations, and batch processing workflows.

```
CREATE OR REPLACE PROCEDURE process_product_restock(
p_product_id INT,
p_added_qty INT
)
LANGUAGE plpgsql AS $$
BEGIN
UPDATE products
SET stock_quantity = stock_quantity + p_added_qty
WHERE product_id = p_product_id;

IF NOT FOUND THEN
RAISE EXCEPTION 'Product Identifier % does not exist.', p_product_id;
END IF;

  INSERT INTO restock_logs (product_id, restock_date, added_quantity)
  VALUES (p_product_id, CURRENT_DATE, p_added_qty);

-- Explicit Transaction Control
COMMIT;
END;
$$;
```

Listing 3: Stored Procedure with Transaction Control

6 Parameter Modes and Default Behavior

Formal parameters in PL/pgSQL are assigned a mode that dictates the direction of data binding between the calling environment and the subprogram. The supported modes are `IN`, `OUT`, and `INOUT`.

6.1 Implicit Default Behavior

If a parameter mode is omitted during the formal parameter declaration, the PL/pgSQL compiler implicitly assigns the `IN` mode by default. Consequently, any unannotated parameter is strictly evaluated as a read-only constant within the subprogram's execution scope; attempting to mutate its state will trigger a compilation error.

6.2 Mode Definitions

- `IN` **Mode:** Specifies that the formal parameter accepts an actual argument from the caller. It is lexically immutable within the subprogram body.
- `OUT` **Mode:** Specifies an uninitialized variable bound to the caller's environment. The subprogram assigns a value to it, which is transmitted back upon termination. When `OUT` parameters are present in a function, the `RETURNS` clause is typically omitted as the function implicitly returns a record of the `OUT` variables.
- `INOUT` **Mode:** A hybrid parameter that accepts an initialized actual argument from the caller, permits state mutation within the subprogram body, and transmits the modified state back to the caller upon termination.

7 Integration of Parameter Modes

The following examples demonstrate the simultaneous application of `IN`, `OUT`, and `INOUT` modes within a single subprogram.

7.1 Function Implementation with Mixed Modes

When a function incorporates `OUT` and `INOUT` parameters, it evaluates to a composite `RECORD` type containing the mutated variables.

```
CREATE OR REPLACE FUNCTION evaluate_customer_loyalty(
p_cust_id IN INT, -- Implicit or explicit IN: Read-only
p_points_accrued INOUT NUMERIC,   -- Mutated and returned
p_eligible_for_upgrade OUT BOOLEAN -- Calculated and returned
)
LANGUAGE plpgsql AS $$
DECLARE
v_current_tier VARCHAR;
BEGIN
-- Read the IN parameter
SELECT loyalty_tier INTO v_current_tier
FROM customers
WHERE customer_id = p_cust_id;

-- Mutate the INOUT parameter based on internal logic
```

```
IF v_current_tier = 'Bronze' AND p_points_accrued < 5000.00 THEN
p_points_accrued := p_points_accrued * 1.10; -- 10 percent point bonus
END IF;

-- Assign the OUT parameter
IF p_points_accrued >= 2000.00 THEN
p_eligible_for_upgrade := TRUE;
ELSE
p_eligible_for_upgrade := FALSE;
END IF;

-- No RETURN statement required; OUT/INOUT parameters form the implicit return
  record
END;
$$;
```

Listing 4: Function integrating IN, OUT, and INOUT modes

Invocation:

```
-- The caller provides arguments only for IN and INOUT parameters.
SELECT * FROM evaluate_customer_loyalty(1, 1500.00);
```

7.2 Procedure Implementation with Mixed Modes

Procedures utilizing mixed parameter modes are invoked via the `CALL` statement. The caller must provide initialized variables for `INOUT` parameters and target variables for `OUT` parameters to capture the modified states.

```
CREATE OR REPLACE PROCEDURE process_category_discount(
p_category IN VARCHAR,-- Read-only constraint
p_discount_budget INOUT NUMERIC,   -- Mutated and returned to caller
p_budget_status OUT VARCHAR -- Assigned and returned to caller
)
LANGUAGE plpgsql AS $$
DECLARE
v_product_count INT;
BEGIN
-- Utilize IN parameter for scalar subquery
SELECT COUNT(*) INTO v_product_count
FROM products
WHERE category = p_category;

-- Transform the INOUT parameter
p_discount_budget := p_discount_budget - (v_product_count * 50.00);

-- Initialize the OUT parameter
IF p_discount_budget > 0 THEN
p_budget_status := 'BUDGET_SURPLUS';
ELSE
p_budget_status := 'BUDGET_DEFICIT';
END IF;

-- Procedural Transaction Control
COMMIT;
END;
$$;
```

Listing 5: Procedure integrating IN, OUT, and INOUT modes

Invocation:

```
DO $$
DECLARE
v_promo_budget NUMERIC := 10000.00; -- Initialized for INOUT
v_status VARCHAR;  -- Uninitialized for OUT
BEGIN
-- Execute the procedure, passing variables by reference
CALL process_category_discount('Electronics', v_promo_budget, v_status);

RAISE NOTICE 'Remaining Budget: %, Status: %', v_promo_budget, v_status;
END;
$$;
```

8 Calling Functions and Procedures

The invocation mechanism in PL/pgSQL is determined by the subprogram classification (*function* versus *procedure*) and the declared parameter modes. Although both constructs encapsulate executable logic, their calling semantics differ significantly in terms of syntactic placement, return handling, and transaction scope.

8.1 Calling Scalar Functions

A scalar function is invoked as part of an expression context. Because it evaluates to a single value, it can appear in a `SELECT` list, `WHERE` clause, `ORDER BY` clause, or be assigned to a variable within a PL/pgSQL block.

```
SELECT calculate_customer_lifetime_value(1);
```

Listing 6: Invoking a Scalar Function in a SELECT Statement

The function may also be integrated into relational queries:

```
SELECT
    customer_id,
    calculate_customer_lifetime_value(customer_id) AS lifetime_value
FROM customers;
```

Within an anonymous block or another function:

```
DO $$
DECLARE
    v_ltv NUMERIC;
BEGIN
    v_ltv := calculate_customer_lifetime_value(1);
    RAISE NOTICE 'Customer LTV: %', v_ltv;
END;
$$;
```

Because functions execute within the transaction context of the invoking statement, their execution is atomic relative to that statement.

8.2 Calling Table-Valued Functions

A table-valued function evaluates to a result set and must therefore be invoked in the `FROM` clause. PostgreSQL treats such functions as virtual tables.

```
SELECT *
FROM some_table_valued_function();
```

Listing 7: Invoking a Table-Valued Function

They may also participate in joins:

```
SELECT c.customer_id, t.*
FROM customers c
JOIN some_table_valued_function() AS t
    ON c.customer_id = t.customer_id;
```

The returned structure conforms to the `RETURNS TABLE` specification or the composite type implied by `OUT` parameters.

8.3 Calling Functions with OUT and INOUT Parameters

When a function declares `OUT` or `INOUT` parameters, PostgreSQL implicitly returns a composite record consisting of those parameters. The caller supplies arguments only for `IN` and `INOUT` parameters.

```
SELECT *
FROM evaluate_customer_loyalty(1, 1500.00);
```

The result set columns correspond to:

- The mutated `INOUT` parameter
- The computed `OUT` parameter

If only a single column is required, it may be projected explicitly:

```
SELECT (evaluate_customer_loyalty(1, 1500.00)).p_eligible_for_upgrade;
```

8.4 Calling Stored Procedures

Stored procedures are invoked using the `CALL` statement. Unlike functions, they cannot be embedded within expressions and do not directly return a value.

```
CALL process_product_restock(10, 25);
```

Listing 8: Basic Procedure Invocation

Because procedures may include explicit transaction control, they execute as independent transactional units when appropriate.

8.5 Calling Procedures with OUT and INOUT Parameters

When invoking procedures with `OUT` or `INOUT` parameters, the caller must supply variables (not literals) for any parameter whose value is returned.

```
DO $$
DECLARE
    v_promo_budget NUMERIC := 10000.00;
    v_status VARCHAR;
BEGIN
    CALL process_category_discount(
        'Electronics',
        v_promo_budget,
        v_status
    );

    RAISE NOTICE 'Remaining Budget: %, Status: %',
        v_promo_budget,
        v_status;
END;
$$;
```

9 Lab Tasks

1. Create a `Scalar Function` named `project_future_revenue` that accepts `p_months_ahead` as an `IN` parameter. Calculate the average daily revenue generated over the last 30 days. Then, calculate the growth rate by comparing the revenue of the last 30 days to the 30 days prior to that. Using the compound interest formula ($A = P(1+r)^t$), project what the total revenue will be for the single 30-day period that occurs exactly `p_months_ahead` in the future. Return this projected numeric value. **Notation:**
   - A: Projected future revenue.
   - P: Present value (the total revenue from the most recent 30 days).
   - r: Growth rate (expressed as a decimal).
   - t: Number of time periods (`p_months_ahead`).
2. Implement a `Stored Procedure` named `apply_bulk_discounts` that takes `p_min_qty` and `p_discount_pct`. Iterate through all `'PENDING'` orders. Within each order, iterate through its `order_items`. If an item's requested `quantity` is $\geq$ `p_min_qty`, calculate the discount, update the item's `unit_price`, and flag that the order's total needs recalculation. After evaluating all items for a single order, if any item was discounted, recalculate and update the order's `total_value`.
3. Design a `Stored Procedure` named `anonymize_inactive_customers` featuring an `OUT` parameter `p_anonymized_count`. Loop through all customers. Calculate the difference in days between `CURRENT_DATE` and their most recent `order_date`. If they have never placed an order, or if their last order was over 730 days (2 years) ago, execute an anonymization process: change their `name` to `'ANONYMIZED'`, `region` to `'UNKNOWN'`, and `loyalty_tier` to `'Bronze'`. Count how many rows were updated.